

이후에 문서별로 재정의됩니다.

, keywordstyle=, commentstyle=, stringstyle=, numberstyle=, numbers=left, numbersep=8pt, breaklines=true, breakatwhitespace=false, frame=single, frameround=ttt, rulecolor=, captionpos=b, showstringspaces=false, tabsize=2, xleftmargin=15pt, xrightmargin=5pt, escapeinside=%%*)

정리[section] [theorem]보조정리 [theorem]명제 [theorem]따름정리 정의[section] 예제[section]

■ 강의명: *CSCI89*

■ 주차: *Lecture 3*

■ 교수명: *UIUC Student*

■ 목적: 본 문서는 *CSCI E-89* 강의의 세 번째 내용을 통합하여 순환 신경망(*RNN*)의 기본 개념부터 자연어 처리(*NLP*)의 핵심 기술까지 다룹니다. 특히 *RNN*의 구조, 파라미터 계산, *LSTM*의 작동 원리, 그리고 텍스트 데이터를 모델에 입력하기 위한 전처리 기법(토큰화, 어간 추출, 표제어 추출, 임베딩)을 상세히 설명합니다. 실제 텍스트 분류 예제를 통해 이론적 지식이 어떻게 적용되는지 보여주며, 비전 공자도 쉽게 이해할 수 있도록 풍부한 설명과 예시를 제공합니다.

Contents

1 용어 정리

다음 표는 본 문서에서 사용되는 주요 용어들을 정리하고 쉽게 설명합니다.

용어	쉬운 설명	원어	비고
순환 신경망	시퀀스(순서가 있는) 데이터를 처리하는 데 특화된 신경망.	<i>Recurrent Neural Network (RNN)</i>	이전 단계의 정보를 기억하여 다음 단계에 활용
장단기 기억망	<i>RNN</i> 의 한 종류로, 장기 의존성 문제를 해결하기 위해 고안된 복잡한 구조.	<i>Long Short-Term Memory (LSTM)</i>	셀 상태(<i>Cell State</i>)와 게이트(<i>Gate</i>)를 통해 정보 흐름 제어
자연어 처리	컴퓨터가 인간의 언어를 이해하고 생성하도록 하는 인공지능 분야.	<i>Natural Language Processing (NLP)</i>	텍스트 분류, 번역, 요약 등 다양한 응용
토큰화	텍스트를 의미 있는 작은 단위(토큰)로 나누는 과정.	<i>Tokenization</i>	단어, 문장, 문자 등이 토큰이 될 수 있음
어간 추출	단어의 접미사나 접두사를 제거하여 단어의 '어간'을 찾는 과정.	<i>Stemming</i>	결과가 실제 단어가 아닐 수 있으며, 자원 축소에 유용
표제어 추출	단어를 사전적 의미를 가지는 '표제어'로 변환하는 과정.	<i>Lemmatization</i>	어간 추출보다 정확하며, 품사 정보 활용
임베딩	단어나 문장 같은 이산적인 데이터를 연속적인 숫자 벡터로 변환하는 기법.	<i>Embedding</i>	고차원 희소 벡터를 저차원 밀집 벡터로 효율적으로 표현
원-핫 인코딩	단어를 고유한 위치에 1을, 나머지는 0을 가지는 벡터로 표현하는 방식.	<i>One-Hot Encoding</i>	희소(<i>Sparse</i>)하며, 단어 간 유사성 표현 어려움
희소 벡터	대부분의 요소가 0인 벡터.	<i>Sparse Vector</i>	메모리 비효율적, 계산 낭비 발생 가능
밀집 벡터	대부분의 요소가 0이 아닌 벡터.	<i>Dense Vector</i>	정보 압축 및 효율적인 계산 가능
<i>OOV</i> 토큰	모델 학습 시 사용된 어휘 사전에 없는 단어를 나타내는 특별한 토큰.	<i>Out-Of-Vocabulary (OOV) Token</i>	사전에 없는 단어 처리 방식 중 하나
드롭아웃	신경망 훈련 중 일부 뉴런을 무작위로 비활성화하여 과적합을 방지하는 기법.	<i>Dropout</i>	모델의 일반화 성능 향상
이진 교차 엔트로피	이진 분류 문제에서 모델의 예측 확률과 실제 레이블 간의 차이를 측정하는 손실 함수.	<i>Binary Cross-Entropy</i>	로지스틱 회귀와 같은 이진 분류 모델에 적합
시그모이드 함수	입력 값을 0과 1 사이의 확률 값으로 변환하는 활성화 함수.	<i>Sigmoid Function</i>	이진 분류 모델의 출력 레이어에 주로 사용

2 핵심 개념·원리

순환 신경망 (RNN) 핵심 요약

*RNN*은 시퀀스 데이터를 처리하는 데 특화된 신경망으로, 이전 단계의 정보를 '기억'하여 현재 단계의 예측에 활용합니다. 이는 텍스트, 음성, 시계열 데이터와 같이 순서가 중요한 데이터에 특히 유용합니다.

2.0.1 RNN의 기본 구조와 작동 원리

*RNN*은 일반적인 신경망(피드포워드 신경망)과 달리, 은닉층의 출력이 다음 시점의 은닉층 입력으로 다시 들어가는 '순환(recurrent)' 연결을 가집니다. 이 순환 연결 덕분에 *RNN*은 시퀀스 데이터의 시간적 의존성을 학습할 수 있습니다.

*RNN*의 직관적 비유: 문장 읽기

우리가 문장을 읽을 때, 각 단어의 의미는 그 앞의 단어들과의 관계 속에서 이해됩니다. 예를 들어 "나는 사과를 먹었다"라는 문장에서 '먹었다'는 '사과'와 연결되어 의미를 완성합니다. *RNN*도 이와 유사하게, '사과'라는 단어를 처리할 때 '나는'이라는 정보와 '사과를'이라는 정보를 함께 고려하여 '먹었다'를 예측하거나 이해합니다.

2.0.2 Simple RNN의 파라미터 계산

RNN 모델의 복잡성을 이해하는 한 가지 방법은 학습 가능한 파라미터(가중치와 바이어스)의 수를 계산하는 것입니다. 이는 모델의 크기와 계산 비용을 가늠하는 데 중요합니다.

강의에서는 `SimpleRNN(2, input_shape = (None, 4)) Keras` .

`SimpleRNN(2)`: 이 순환 레이어에는 2개의 뉴런(또는 은닉 상태)이 있습니다. 이는 출력 y 벡터가 2차원임을 의미합니다.

`input_shape = (None, 4)` :

`None`: 시간 스텝(*time steps*)의 수는 가변적임을 의미합니다. 훈련 중 네트워크가 자동으로 시간 스텝 수를 파악합니다.

`4`: 각 시간 스텝에서 입력 x 벡터의 차원이 4임을 의미합니다. 즉, 각 시점에 4개의 특성(*feature*)이 입력됩니다.

SimpleRNN 파라미터 계산 공식

SimpleRNN 레이어의 총 파라미터 수는 다음과 같이 계산됩니다:

총 파라미터 수 = (입력 특성 수 × 은닉 뉴런 수) + (은닉 뉴런 수 × 은닉 뉴런 수) + 은닉 뉴런 수

여기서:

- 입력 특성 수: 각 시간 스텝에서 들어오는 입력 벡터의 차원 (예: 4).
- 은닉 뉴런 수: RNN 레이어 내의 뉴런 수 (예: 2).
- 은닉 뉴런 수 (세 번째 항): 각 뉴런에 대한 바이어스(bias) 수.

2.0.3 RNN의 다양한 입출력 아키텍처

RNN은 시퀀스 데이터를 처리하는 방식에 따라 다양한 입출력 구조를 가질 수 있습니다. Keras에서는 `return_sequences`를 사용하여 다양한 입출력 구조를 구현합니다.

- 다대다 (Many-to-Many): 입력 시퀀스에 대해 출력 시퀀스를 생성합니다.
 - 예시: 기계 번역 (입력 문장 → 출력 문장).
 - `return_sequences = True`
- 다대일 (Many-to-One): 입력 시퀀스에 대해 단일 출력을 생성합니다.
 - 예시: 감성 분석 (입력 문장 → 긍정/부정).
 - `return_sequences = False`
- 일대다 (One-to-Many): 단일 입력에 대해 출력 시퀀스를 생성합니다.
 - 예시: 이미지 캡셔닝 (입력 이미지 → 이미지 설명 문장).
 - 이 경우, 단일 입력 후에는 '가짜(fake) 0 벡터'를 입력으로 넣어 시퀀스를 생성합니다.

주의: RNN의 출력과 `return_sequences`

RNN 레이어는 기본적으로 각 시간 스텝마다 출력을 생성합니다. `return_sequences = True`를 사용하면, 입력 시퀀스의 길이와 동일한 출력 시퀀스를 생성합니다.

2.1 인코더-디코더 (Encoder-Decoder) 아키텍처

인코더-디코더 핵심 요약

인코더-디코더는 입력 시퀀스를 고정된 크기의 '컨텍스트 벡터'로 압축하는 인코더와, 이 컨텍스트 벡터로부터 출력 시퀀스를 생성하는 디코더로 구성됩니다. 이는 정보 압축과 변환에 유용합니다.

2.1.1 정보 압축을 위한 병목 (Bottleneck)

인코더-디코더는 오토인코더(Autoencoder)의 한 형태로 볼 수 있으며, 특히 '병목(bottleneck)' 구조를 통해 정보 압축을 수행합니다.

인코더-디코더의 비유: 요약과 확장

긴 문장(입력 시퀀스)을 읽고 핵심 아이디어(컨텍스트 벡터)를 한 문장으로 요약한 다음, 그 핵심 아이디어로부터 원래 문장과 유사하거나 다른 언어의 문장(출력 시퀀스)을 다시 작성하는 것과 같습니다. 핵심 아이디어가 바로 '병목'을 통과한 압축된 정보입니다.

- 오토인코더 (Autoencoder): 입력 데이터를 자기 자신으로 매핑하는 신경망입니다. 이 과정에서 중간에 저차원의 '병목' 레이어를 두어 데이터의 중요한 특징을 압축하여 학습합니다.
 - 노이즈 제거 오토인코더 (Denoising Autoencoder): 손상된 이미지나 노이즈가 포함된 텍스트를 입력으로 받아 원본 데이터를 복원하도록 훈련됩니다. 병목 레이어는 노이즈와 같은 중요하지 않은 정보를 걸러내고 핵심 정보만 압축하는 방법을 학습합니다.
- 번역에서의 활용: RNN 기반 인코더-디코더는 2014년부터 2017년까지 기계 번역의 지배적인 기술이었습니다.
 - 인코더가 원어 문장(시퀀스)을 하나의 컨텍스트 벡터로 압축합니다.
 - 디코더가 이 컨텍스트 벡터로부터 대상 언어의 문장(시퀀스)을 생성합니다.
 - 한계: 하나의 고정된 컨텍스트 벡터가 긴 문장의 모든 정보를 담기 어렵다는 '정보 병목' 문제가 있었습니다. 이는 나중에 어텐션 메커니즘(Attention Mechanism)을 사용하는 트랜스포머(Transformer) 모델로 해결됩니다.

2.2 장단기 기억망 (Long Short-Term Memory, LSTM)의 이해

LSTM 핵심 요약

LSTM은 RNN의 '장기 의존성(long-term dependency)' 문제를 해결하기 위해 고안된 특수한 RNN 구조입니다. '셀 상태(Cell State)'와 '게이트(Gate)' 메커니즘을 통해 정보를 선택적으로 기억하거나 잊어버리면서 장기적인 패턴을 학습할 수 있습니다.

2.2.1 LSTM의 두 가지 기억: 셀 상태 (C)와 은닉 상태 (H)

LSTM은 두 가지 주요 '기억'을 가집니다.

- 셀 상태 (Cell State, C): 장기 기억의 역할을 합니다. 정보가 셀 상태를 따라 흐르면서 게이트에 의해 추가되거나 제거됩니다.
- 은닉 상태 (Hidden State, H): 단기 기억의 역할을 하며, 현재 시점의 출력과 다음 시점의 입력으로 사용됩니다.

C와 H의 차원 관계

LSTM 구조 내에서 셀 상태 (C)와 은닉 상태 (H)는 항상 동일한 차원 (*dimension*)을 가집니다. 이는 게이트를 통한 연산(요소별 곱셈, 덧셈)이 차원을 변경하지 않기 때문입니다.

C와 H의 차원 관계 직관적 이해

LSTM 셀 내부의 연산들을 살펴보면, 셀 상태 (C)와 은닉 상태 (H)는 서로 더해지거나(*summation*), 요소별로 곱해지는(*element-wise multiplication*) 방식으로 상호작용합니다. 벡터의 덧셈이나 요소별 곱셈은 벡터의 차원을 변경하지 않습니다. 따라서, C 와 H 는 항상 같은 차원을 유지해야 합니다. 예를 들어, *LSTM*(5)라고 지정하면 C 와 H 모두 5차원 벡터가 됩니다.

2.2.2 LSTM의 게이트 메커니즘

*LSTM*은 세 가지 주요 게이트를 통해 셀 상태의 정보 흐름을 제어합니다. 각 게이트는 시그모이드 활성화 함수를 사용하여 0과 1 사이의 값을 출력하며, 이는 정보의 '얼마나'를 통과시킬지 결정합니다.

- 망각 게이트 (*Forget Gate*): 이전 셀 상태에서 어떤 정보를 '잊어버릴지' 결정합니다.
- 입력 게이트 (*Input Gate*): 현재 입력과 이전 은닉 상태에서부터 어떤 새로운 정보를 '셀 상태에 추가할지' 결정합니다.
- 출력 게이트 (*Output Gate*): 셀 상태를 기반으로 현재 시점의 은닉 상태 (H)와 출력 (Y)을 '얼마나' 내보낼지 결정합니다.

각 게이트는 독립적인 완전 연결(*Fully Connected, Dense*) 서브 네트워크로 구성되며, *LSTM*(N)에서 N 은 각 서브 네트워크의 뉴런 수를 의미합니다. 따라서 *LSTM*(N)은 총 $4 \times N$ 개의 뉴런을 내부적으로 가집니다.

2.3 양방향 순환 신경망 (Bidirectional RNN/LSTM)

양방향 RNN/LSTM 핵심 요약

양방향 *RNN/LSTM*은 입력 시퀀스를 순방향과 역방향으로 모두 처리하여, 각 시점의 출력에 대해 과거와 미래의 모든 문맥 정보를 활용할 수 있도록 합니다. 이는 특히 텍스트와 같이 전체 문맥이 중요한 데이터에 유용합니다.

2.3.1 양방향 처리의 필요성

일반적인 *RNN*은 과거의 정보만을 기반으로 현재를 예측합니다. 하지만 많은 시퀀스 데이터, 특히 자연어에서는 미래의 정보(문장의 뒷부분)가 현재의 의미를 이해하는 데 중요할 수 있습니다.

양방향 RNN의 비유: 문장 전체 읽기

독일어와 같이 문장의 부정어가 마지막에 오는 언어의 경우, 문장의 끝을 읽어야만 전체 문장이 긍정적인지 부정적인지 정확히 판단할 수 있습니다. 순방향으로만 읽는다면 문장 끝에 도달하기 전까지는 정확한 의미를 파악하기 어렵습니다. 양방향 RNN은 문장을 앞에서부터 읽고(순방향), 뒤에서부터도 읽어(역방향) 두 정보를 모두 활용하여 더 풍부한 문맥 이해를 가능하게 합니다.

2.3.2 구조와 파라미터 계산

양방향 RNN/LSTM은 두 개의 독립적인 RNN/LSTM 레이어를 가집니다. 하나는 입력 시퀀스를 순방향으로 처리하고, 다른 하나는 역방향으로 처리합니다. 두 레이어의 출력은 최종적으로 연결(concatenate)되어 다음 레이어의 입력으로 사용됩니다.

양방향 RNN/LSTM의 파라미터 및 출력 차원

- 파라미터 수: 양방향 레이어는 두 개의 독립적인 단방향 레이어를 포함하므로, 단방향 레이어의 파라미터 수의 정확히 두 배가 됩니다.
- 출력 차원: 각 단방향 레이어의 출력이 연결되므로, 양방향 레이어의 출력 차원 또한 단방향 레이어 출력 차원의 두 배가 됩니다.

양방향 LSTM 파라미터 계산 예시

만약 $LSTM(128)$ 레이어가 8320 개의 파라미터를 가진다면, $Bidirectional(LSTM(128))$ 레이어는 $8320 \times 2 = 16640$ 개의 파라미터를 가집니다.

이후에 연결되는 Dense 레이어의 파라미터는 단순히 두 배가 되지 않을 수 있습니다. 예를 들어, $Dense(1)$ 레이어는 단일 바이어스를 가지므로, 양방향 LSTM의 출력이 2배가 되어 입력이 2배가 되더라도 바이어스는 하나만 추가되어 파라미터가 $32 \times 2 + 1 = 65$ 와 같이 계산됩니다.

2.4 자연어 처리 (Natural Language Processing, NLP) 개요

NLP 핵심 요약

NLP는 컴퓨터가 인간의 자연어를 이해하고, 해석하고, 생성할 수 있도록 하는 인공지능 분야입니다. 언어학적 지식과 인공지능 기술을 결합하여 다양한 언어 관련 문제를 해결합니다.

2.4.1 AI, 머신러닝, NLP의 관계

- 인공지능 (Artificial Intelligence, AI): 인간의 지능을 모방하는 광범위한 분야입니다. 규칙 기반 시스템(하드코딩된 규칙)부터 복잡한 학습 알고리즘까

지 모두 포함합니다.

- 예시: 의사가 특정 증상에 대한 처방 규칙을 컴퓨터에 입력하여 진단하는 시스템.
- 머신러닝 (*Machine Learning, ML*): 데이터와 결과(레이블)를 입력으로 받아 '규칙'을 스스로 학습하는 AI의 하위 분야입니다.
 - 예시: 선형 회귀 모델이 데이터로부터 최적의 β_0, β_1 계수를 학습하여 예측 규칙을 생성하는 것.
- 자연어 처리 (*NLP*): 언어학(언어의 구조, 의미)과 인공지능(특히 머신러닝, 딥러닝)이 결합된 다학제 분야입니다. 컴퓨터가 인간 언어를 이해하고 생성하도록 하는 것이 목표입니다.
 - NLP는 때로는 언어학적 규칙(예: 단어를 어근 형태로 줄이는 규칙)을 하드코딩하여 사용하기도 하므로, 순수 머신러닝의 범주를 넘어 AI의 더 넓은 영역에 걸쳐 있습니다.

2.4.2 NLP의 도전 과제

인간의 언어는 모호하고, 문맥에 따라 의미가 달라지며, 배경 지식을 요구하는 경우가 많아 컴퓨터가 이해하기 매우 어렵습니다.

- 모호성 (*Ambiguity*): "Court to try shooting defendant." (법원이 피고를 총살하려 한다? 아니면 법원이 피고의 총격 사건을 심리한다?)와 같이 문장의 해석이 여러 가지일 수 있습니다.
- 배경 지식 (*Background Knowledge*): 특정 분야의 전문 지식이나 상식이 없으면 문장의 의미를 정확히 파악하기 어렵습니다.

2.4.3 NLP의 주요 응용 분야

- 텍스트 분류 (*Text Classification*): 스팸 메일 감지, 뉴스 기사 주제 분류, 감성 분석 (긍정/부정).
- 개체명 인식 (*Named Entity Recognition, NER*): 텍스트에서 사람 이름, 조직, 장소 등 고유한 개체를 식별합니다. (예: 번역 시 고유명사는 번역하지 않고 그대로 두는 것이 중요).
- 감성 분석 (*Sentiment Analysis*): 텍스트에 담긴 감정(긍정, 부정, 중립)을 파악합니다. (예: 영화 리뷰 분석). 비꼬는 표현(*sarcasm*) 등은 매우 어려운 문제입니다.
- 정보 검색 (*Information Retrieval*): 대규모 텍스트 데이터에서 관련성 높은 정보를 찾아냅니다. (예: 검색 엔진). *TF-IDF*, *BM25* 등의 기법이 사용됩니다.
- 광학 문자 인식 (*Optical Character Recognition, OCR*): 이미지 형태의 텍스트를 컴퓨터가 편집 가능한 텍스트로 변환합니다. (예: 문서 스캔).
- 기계 번역 (*Machine Translation*): 한 언어의 문장을 다른 언어로 번역합니다. (예: 구글 번역).
- 텍스트 요약 (*Text Summarization*): 긴 텍스트의 핵심 내용을 추출하여 짧은

요약문을 생성합니다.

- 음성 인식 (*Speech Recognition*): 음성 데이터를 텍스트로 변환합니다. (예: 음성 비서).
- 질의응답 시스템 (*Question Answering Systems*): 질문에 대한 답변을 텍스트에서 찾아 제공합니다. (예: "프랑스의 수도는?").
- 챗봇 (*Chatbots*): 사용자와 대화하며 질문에 답하거나 작업을 수행합니다. (예: ChatGPT). 질의응답 시스템과 유사하지만 대화의 연속성이 중요합니다.
- 토픽 모델링 (*Topic Modeling*): 텍스트 문서 집합에서 숨겨진 주제(*topic*)를 찾아내고, 각 문서가 어떤 주제로 구성되어 있는지 분석합니다. 비지도 학습의 일종입니다.
- 임상 응용 (*Clinical Applications*): 의료 문헌 분석, 증상 기반 질병 진단 지원 등 의료 분야에서의 NLP 활용.
- 언어 생성 (*Language Generation*): 인간과 유사한 자연스러운 텍스트를 생성합니다.

3 절차/방법: 텍스트 전처리 및 모델 구축

3.1 텍스트 전처리 (Text Preprocessing) 과정

신경망에 텍스트를 입력하기 전에, 텍스트를 컴퓨터가 이해할 수 있는 형태로 변환하는 전처리 과정이 필수적입니다. 주요 단계는 토큰화, 어간 추출, 표제어 추출입니다.

3.1.1 토큰화 (Tokenization)

토큰화 핵심 요약

토큰화는 텍스트를 의미 있는 최소 단위인 '토큰(token)'으로 분할하는 과정입니다. 일반적으로 단어가 토큰이 되지만, 문장, 문자, N -그램 등 다양한 단위로 토큰화할 수 있습니다.

- 정의: 텍스트를 개별 단위(토큰)로 분리하는 과정입니다.
- 토큰의 종류:
 - 단어 토큰화 (Word Tokenization): 가장 일반적인 형태로, 텍스트를 개별 단어로 나눕니다. 구두점도 별도의 토큰으로 처리될 수 있습니다.
 - 문장 토큰화 (Sentence Tokenization): 텍스트를 개별 문장으로 나눕니다.
 - N -그램 토큰화 (N -gram Tokenization): N 개의 연속된 단어를 하나의 토큰으로 간주합니다 (예: 2-그램은 두 단어).
 - 문자 토큰화 (Character Tokenization): 텍스트를 개별 문자로 나눕니다.
 - 서브워드 토큰화 (Subword Tokenization): 단어를 더 작은 의미 단위(서브워드)로 분할합니다. (예: 'un-happy'를 'un'과 'happy'로).
- 불용어 (Stop Words): 'the', 'a', 'is'와 같이 자주 등장하지만 텍스트의 핵심 의미를 파악하는 데 크게 기여하지 않는 단어들입니다. 텍스트 분류와 같은 작업에서는 불용어를 제거하는 것이 모델 성능 향상에 도움이 될 수 있습니다.

NLTK를 이용한 토큰화 예시

NLTK (Natural Language Toolkit) 라이브러리는 파이썬에서 텍스트를 토큰화하는 데 널리 사용됩니다.

```
1 import nltk
2 from nltk.tokenize import word_tokenize, sent_tokenize
3
4 text = "Henry Ford's innovation profoundly impacted the automotive
5         industry. He revolutionized mass production."
6
7 # 단어토큰화
8 word_tokens = word_tokenize(text)
9 print("단어 토큰:", word_tokens)
```

```

9 # 출력: ['Henry', 'Ford', "'s", 'innovation', 'profoundly', 'impacted', '
    the', 'automotive', 'industry', '.', 'He', 'revolutionized', 'mass',
    'production', '.']
10
11 # 문장토큰화
12 sent_tokens = sent_tokenize(text)
13 print("문장 토큰:", sent_tokens)
14 # 출력: ["Henry Ford's innovation profoundly impacted the automotive
    industry.", "He revolutionized mass production."]

```

Listing 1: NLTK 단어 토큰화 예시

라이브러리 사용 권장

직접 토큰화 로직을 구현하는 것보다 *NLTK*와 같은 라이브러리를 사용하는 것이 좋습니다. 라이브러리는 'can't'와 같은 축약형이나 구두점 처리 등 복잡한 예외 상황을 더 잘 처리합니다.

3.1.2 어간 추출 (Stemming)

어간 추출 핵심 요약

어간 추출은 단어의 접미사나 접두사를 제거하여 단어의 '어간(*stem*)'을 찾는 과정입니다. 이는 단어의 다양한 형태를 하나의 공통된 형태로 줄여 어휘 사전을 축소하고 모델의 차원을 줄이는 데 사용됩니다.

- 정의: 단어의 형태학적 변형(예: *running*, *runs*, *ran*)을 무시하고 단어의 '어간'을 추출하는 기법입니다. 주로 접미사를 제거하는 방식으로 작동합니다.
- 목적: 어휘 사전에 포함될 단어의 수를 줄여 모델의 복잡성을 낮춥니다.
- 특징: 어간 추출의 결과는 종종 실제 사전에 없는 단어 형태일 수 있습니다. (예: 'electric' -> 'electr', 'octopus' -> 'octop').
- 종류:
 - *Porter Stemmer* (포터 스테머): 1979년에 개발된 가장 유명한 어간 추출 알고리즘 중 하나입니다. 규칙 기반으로 접미사를 제거합니다. (예: 'transportation' -> 'transport', 'electric' -> 'electr').
 - *Snowball Stemmer* (스노볼 스테머): *Porter Stemmer*의 개선 버전으로, 여러 언어를 지원하며 정확도가 향상되었습니다.
 - *Lancaster Stemmer* (랭커스터 스테머): *Porter Stemmer*보다 더 공격적으로 단어를 자릅니다.

NLTK를 이용한 어간 추출 예시

```

1 from nltk.stem import PorterStemmer, SnowballStemmer
2 from nltk.tokenize import word_tokenize

```

```

3
4 text = "Henry Ford's innovation profoundly impacted the automotive
      industry. He revolutionized mass production."
5 tokens = word_tokenize(text)
6
7 porter_stemmer = PorterStemmer()
8 snowball_stemmer = SnowballStemmer("english") # 언어지정
9
10 stemmed_porter = [porter_stemmer.stem(token) for token in tokens]
11 stemmed_snowball = [snowball_stemmer.stem(token) for token in tokens]
12
13 print("원본 토큰:", tokens)
14 print("Porter Stemmer 결과:", stemmed_porter)
15 # 예시출력
16     : ['henri', 'ford', "'s", 'innov', 'profoundli', 'impact', 'the', '
17     automot', 'industri', '.', 'he', 'revolut', 'mass', 'product', '.']
18 print("Snowball Stemmer 결과:", stemmed_snowball)
19 # 예시출력 : ['henri', 'ford', "'s", 'innov', 'profound', 'impact', 'the',
20     'automot', 'industri', '.', 'he', 'revolut', 'mass', 'product', '.']

```

Listing 2: NLTK Porter Stemmer 예시

어간 추출의 특징

'Henry'가 'Henri'로, 'profoundly'가 'profoundli' 또는 'profound'로 변하는 것을 볼 수 있습니다. 이는 어간 추출이 단어를 사전형으로 만드는 것이 아니라, 단순히 규칙에 따라 접미사를 제거하여 '어간'을 찾는 과정이기 때문입니다. 결과가 실제 사전에 없는 단어일 수 있습니다.

3.1.3 표제어 추출 (Lemmatization)

표제어 추출 핵심 요약

표제어 추출은 단어를 사전적 의미를 가지는 '표제어(lemma)'로 변환하는 과정입니다. 어간 추출보다 더 정교하며, 단어의 품사(명사, 동사 등) 정보를 활용하여 정확한 원형을 찾습니다.

- 정의: 단어의 다양한 형태를 문법적 규칙에 따라 '표제어(lemma)' 또는 사전형으로 변환하는 기법입니다. (예: 'driving', 'drove', 'driven' -> 'drive').
- 목적: 어간 추출과 마찬가지로 차원 축소를 목표로 하지만, 결과가 항상 실제 사전에 있는 단어 형태를 유지합니다.
- 특징: 어간 추출보다 계산 비용이 더 많이 듭니다. 단어의 품사(Part-of-Speech, POS) 정보를 먼저 식별해야 하는 경우가 많습니다.
- 도구:
 - WordNetLemmatizer (NLTK): NLTK에서 제공하는 표제어 추출 도구입니다. 품

사 태깅이 필요하며, 주로 영어에 사용됩니다. 학술 연구에서 인기가 많습니다.

- SpaCy: 산업 환경에서 많이 사용되는 라이브러리로, WordNetLemmatizer보다 빠르고 효율적입니다.

NLTK와 SpaCy를 이용한 표제어 추출 예시

```

1 import nltk
2 from nltk.stem import WordNetLemmatizer
3 from nltk.tokenize import word_tokenize
4 # nltk.download('wordnet')
5 # nltk.download('omw-1.4')
6 # nltk.download('averaged_perceptron_tagger') # 품사태깅을위해필요
7
8 import spacy
9 # python -m spacy download en_core_web_sm # SpaCy 모델다운로드
10
11 text = "Henry Ford's innovation profoundly impacted the automotive
12         industry. He revolutionized mass production."
13
14 # NLTK WordNetLemmatizer
15 lemmatizer = WordNetLemmatizer()
16 lemmatized_nltk = [lemmatizer.lemmatize(token, pos='v') for token in
17                     tokens] # 동사(v)로 가
18                             정
19 # 실제사용시에는품사태깅후해당품사에맞는 pos 인자를전달해야합니다 .
20 # 예: nltk.pos_tag(tokens) 후각단어의품사를기반으로 lemmatize 호출
21
22 print("NLTK Lemmatizer 동사( 가정) 결과:", lemmatized_nltk)
23 # 예시출력 : ['Henry', 'Ford', "'s", 'innovation', 'profoundly', 'impact',
24               'the', 'automotive', 'industry', '.', 'He', 'revolutionize', 'mass',
25               'production', '.']
26
27 # SpaCy Lemmatization
28 nlp = spacy.load("en_core_web_sm")
29 doc = nlp(text)
30 lemmatized_spacy = [token.lemma_ for token in doc]
31
32 print("SpaCy Lemmatizer 결과:", lemmatized_spacy)
33 # 예시출력 : ['Henry', 'Ford', "'s", 'innovation', 'profoundly', 'impact',
34               'the', 'automotive', 'industry', '.', 'he', 'revolutionize', 'mass',
35               'production', '.']

```

Listing 3: NLTK WordNetLemmatizer 및 SpaCy 예시

어간 추출 vs. 표제어 추출

- 어간 추출 (Stemming): 'driving' -> 'driv'. 빠르고 간단하지만, 결과가 실제 단어가 아닐 수 있습니다.
 - 표제어 추출 (Lemmatization): 'driving' -> 'drive'. 느리고 복잡하지만, 결과가 항상 실제 사전에 있는 단어입니다.
- 어떤 방법을 사용할지는 작업의 목적과 요구되는 정확도에 따라 달라집니다. 텍스트 분류와 같이 의미의 정확한 복원보다 차원 축소가 중요한 경우에는 어간 추출도 충분히 효과적일 수 있습니다.

3.2 텍스트 분류 모델 구축 (실습)

강의에서는 20 Newsgroups 데이터셋을 사용하여 텍스트 분류 모델을 구축하는 과정을 설명했습니다. 목표는 게시물의 내용을 기반으로 '하키(hockey)' 관련 게시물인지 '판매(for sale)' 관련 게시물인지 분류하는 것입니다.

3.2.1 데이터셋 준비

- 데이터셋: `sklearn.datasets.fetch_20newsgroups` 20 'rec.sport.hockey' 'misc.forsale'
- 분리: 훈련(train) 데이터셋과 테스트(test) 데이터셋으로 나눕니다.
- 레이블: 'for sale'은 0, 'hockey'는 1로 레이블링됩니다.

3.2.2 텍스트를 숫자로 변환: 임베딩 (Embedding)

임베딩 레이어 핵심 요약

임베딩 레이어는 단어와 같은 이산적인(discrete) 데이터를 저차원의 연속적인(continuous) 숫자 벡터로 변환하는 신경망 레이어입니다. 이는 고차원의 희소(sparse)한 원-핫 인코딩의 비효율성을 해결하고, 단어 간의 의미적 유사성을 벡터 공간에 반영하여 모델의 학습 효율과 성능을 높입니다.

- 원-핫 인코딩의 한계:
 - 희소성 (Sparsity): 어휘 사전의 크기가 커질수록 벡터의 대부분이 0이 되어 메모리 낭비가 심합니다. (예: 10,000개 단어 사전에서 각 단어는 10,000 차원 벡터).
 - 의미 관계 부족: 단어 간의 유사성이나 관계를 표현할 수 없습니다. 모든 단어는 서로 독립적으로 간주됩니다.
- 임베딩의 필요성:
 - 차원 축소 (Dimensionality Reduction): 고차원의 희소 벡터를 저차원의 밀집(dense) 벡터로 변환하여 효율성을 높입니다.

- 의미 학습 (Semantic Learning): 학습 가능한 파라미터(가중치)를 통해 단어의 의미적 유사성을 벡터 공간에 반영합니다. (예: '왕'과 '여왕'은 벡터 공간에서 가까이 위치).
- 작동 원리:
 - 임베딩 레이어는 입력으로 단어의 인덱스(정수)를 받습니다.
 - 내부적으로는 각 단어 인덱스에 해당하는 '임베딩 벡터'를 가집니다. 이 벡터들은 모델 훈련 과정에서 함께 학습됩니다.
 - $Embedding(input_{dim}, output_{dim})$:
 - $input_{dim}$: (+ OOV).
 - $output_{dim}$: (: 128).

임베딩의 비유: 사람의 특징 벡터

사람을 '남성'과 '여성'으로 분류할 때, 원-핫 인코딩은 '남성'을 $[1, 0]$, '여성'을 $[0, 1]$ 로 표현합니다. 이는 두 범주가 완전히 독립적임을 나타냅니다.

하지만 임베딩은 각 범주에 대해 학습 가능한 숫자(예: 남성 $\rightarrow 0.8$, 여성 $\rightarrow 0.2$)를 할당하여, 이 숫자가 모델의 다른 부분에서 의미 있는 방식으로 사용되도록 합니다. 더 복잡한 경우, '성별', '나이', '직업' 등 여러 특징을 나타내는 벡터로 사람을 표현할 수 있습니다. 임베딩은 이러한 특징 벡터를 데이터로부터 스스로 학습합니다.

임베딩 레이어의 효율성

수학적으로 임베딩 레이어는 선형 활성화 함수와 바이어스가 없는 완전 연결(Dense) 레이어와 유사하게 볼 수 있습니다. 그러나 실제 구현에서는 단어 인덱스를 직접 사용하여 해당 임베딩 벡터를 '조회(lookup)'하는 방식으로 작동하므로, 고차원 원-핫 벡터에 대한 행렬 곱셈을 수행하는 것보다 훨씬 계산적으로 효율적입니다.

3.2.3 과적합 방지: 드롭아웃 (Dropout)

드롭아웃 핵심 요약

드롭아웃은 신경망의 과적합(overfitting)을 방지하기 위한 정규화(regularization) 기법입니다. 훈련 과정에서 무작위로 일부 뉴런의 연결을 끊어 모델이 특정 특징에 과도하게 의존하는 것을 막고 일반화 성능을 향상시킵니다.

- 정의: 훈련 단계에서 각 미니 배치(mini-batch)마다 무작위로 특정 비율의 뉴런을 비활성화(출력을 0으로 설정)합니다.
- 작동 방식:
 - $Dropout(0.2)$: 각 뉴런이 20%의 확률로 비활성화됩니다.
 - 비활성화된 뉴런은 해당 훈련 스텝에서 가중치 업데이트에 참여하지 않습니다.
 - 매 훈련 스텝마다 무작위로 다른 뉴런들이 비활성화되므로, 모델은 다양한 '하위 네트워크'를 학습하는 효과를 얻습니다.
- 종류:
 - 일반 드롭아웃: 입력 레이어나 LSTM의 출력에 적용되어 특정 특성 값들을 0으로 만듭니

다.

- 순환 드롭아웃 (*Recurrent Dropout*): *LSTM* 내부의 순환 연결에 적용되어 은닉 상태의 일부를 0으로 만듭니다.

드롭아웃의 비유: 팀워크 훈련

한 팀의 모든 구성원이 항상 완벽하게 제 역할을 한다고 가정하면, 특정 구성원이 없으면 팀 전체가 무너질 수 있습니다. 드롭아웃은 훈련 중에 무작위로 팀원 몇 명을 쉬게 하는 것과 같습니다. 이렇게 하면 나머지 팀원들은 어떤 상황에서도 팀의 목표를 달성하는 방법을 학습하게 되어, 특정 개인에게 의존하지 않는 더 강하고 유연한 팀(모델)이 됩니다.

3.2.4 모델 아키텍처

강의에서 제시된 텍스트 분류 모델은 다음과 같은 간단한 *LSTM* 기반 구조를 가집니다.

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Embedding, LSTM, Dropout, Dense
3
4 def build_model(max_features, embedding_dim, max_len):
5     model = Sequential()
6     # 1. 임베딩레이어 : 단어인덱스를밀집벡터로변환
7     model.add(Embedding(input_dim=max_features + 1, output_dim=embedding_dim,
8                          input_length=max_len))
9     # 2. 드롭아웃: 임베딩레이어출력에적용하여과적합방지
10    model.add(Dropout(0.2))
11    # 3. LSTM 레이어: 시퀀스데이터의장기의존성학습
12    model.add(LSTM(128)) # 개의128 은닉상태
13    # 4. 드롭아웃: LSTM 레이어출력에적용하여과적합방지
14    model.add(Dropout(0.2))
15    # 5. 완전연결레이어 : 최종분류를위한 Dense 레이어
16    # 이진분류이므로개 1 뉴런과시그모이드활성화함수사용
17    model.add(Dense(1, activation='sigmoid'))
18
19    # 모델컴파일
20    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['
    accuracy'])
21    return model

```

Listing 4: 텍스트 분류 모델 아키텍처

- *Embedding Layer*: $\text{max_features} + 1$ (+ OOV)
- *Dropout Layer*: 임베딩 레이어의 출력에 적용됩니다.
- *LSTM Layer*: 128개의 은닉 상태를 가지는 *LSTM* 레이어입니다. *return_sequences*
- *Dropout Layer*: *LSTM* 레이어의 출력에 적용됩니다.
- *Dense Layer*: 1개의 뉴런과 *sigmoid* 활성화 함수를 사용하여 이진 분류를 수행합니다.

3.2.5 손실 함수 (Loss Function)와 활성화 함수 (Activation Function)

- 시그모이드 활성화 함수 (*Sigmoid Activation Function*):
 - 정의: 입력 값을 0과 1 사이의 값으로 변환하는 함수입니다.
 - 공식: $\sigma(z) = \frac{1}{1+e^{-z}}$
 - 활용: 이진 분류 문제에서 출력 레이어에 사용되어 예측 확률을 나타냅니다. 로지스틱 회귀 (*Logistic Regression*)의 핵심입니다.
- 이진 교차 엔트로피 (*Binary Cross-Entropy*):
 - 정의: 이진 분류 문제에서 모델의 예측 확률과 실제 레이블(0 또는 1) 간의 차이를 측정하는 손실 함수입니다.
 - 공식: $L = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$
 - 활용: 시그모이드 활성화 함수와 함께 이진 분류 모델의 손실 함수로 사용됩니다.
 - 주의: 이진 분류 문제에서 평균 제곱 오차(*Mean Squared Error, MSE*)는 적절하지 않습니다. *MSE*는 확률 분포가 아닌 연속적인 값의 예측에 더 적합하며, 이진 분류에서는 경사 하강법의 수렴을 방해하는 등의 문제가 발생할 수 있습니다.

3.2.6 데이터 전처리 상세 과정

모델에 텍스트를 입력하기 위해 다음 단계를 거칩니다.

- 최대 특성 수 (*max_features*): 어휘 사전에 포함할 가장 빈번한 단어의 수를 지정합니다 (예: 10,000개). 이는 모델의 복잡성을 제어하고 희소성을 줄입니다.
- 최대 길이 (*max_len*): 각 텍스트 시퀀스의 최대 길이를 지정합니다 (예: 500). 길이가 긴 텍스트는 잘리고, 짧은 텍스트는 0으로 채워집니다(패딩).
- OOV (*Out-Of-Vocabulary*) 토큰: $\text{max_features} + 1$ ~ $\text{max_features} + \text{OOV_count}$ 범위.
- 단어 빈도 계산 및 인덱싱:
 1. 훈련 텍스트를 토큰화합니다.
 2. 각 토큰의 빈도를 계산합니다.
 3. 빈도 순으로 정렬하여 상위 max_features 토큰을 선택합니다.
 3. 각 단어에 고유한 인덱스를 부여합니다 (OOV는 0, 나머지는 1부터 시작).
- 텍스트를 인덱스 시퀀스로 변환: 각 텍스트를 해당 단어 인덱스의 시퀀스로 변환합니다. 사전에 없는 단어는 OOV 인덱스(0)로 대체됩니다.

텍스트를 인덱스 시퀀스로 변환 예시

원본 텍스트: "This is a test sentence." 어휘 사전: 'this': 1, 'is': 2, 'a': 3, 'test': 4, 'sentence': 5, 'OOV': 0

변환된 인덱스 시퀀스: [1, 2, 3, 4, 5]

만약 "new word here"라는 텍스트가 들어오고 'new', 'word', 'here'가 사전에 없다면:
변환된 인덱스 시퀀스: [0, 0, 0]

3.2.7 모델 훈련 및 평가

- **훈련:** `model.fit()` 함수를 사용하여 훈련 데이터와 레이블로 모델을 훈련합니다. `epochs` (훈련 반복 횟수)와 `batch_size`() .
- **평가:** `model.evaluate()` 함수를 사용하여 테스트 데이터셋에서 모델의 성능을 평가합니다. `accuracy` (정확도)는 분류 문제에서 모델의 예측이 얼마나 정확한지를 나타내는 지표입니다.

전처리 기법별 성능 비교 (예시)

강의에서 제시된 결과는 다음과 같습니다.

- 토큰화만 적용: 테스트 정확도 약 **93.5%**
- 토큰화 + 어간 추출 적용: 테스트 정확도 약 **96.8%**
- 토큰화 + 표제어 추출 적용: 테스트 정확도 약 **93.8%**

이 예시에서는 어간 추출이 가장 좋은 성능을 보였습니다. 이는 어간 추출이 어휘 사전을 효과적으로 줄여 모델이 더 일반적인 패턴을 학습하도록 도왔기 때문일 수 있습니다. 하지만 이는 데이터셋과 모델에 따라 달라질 수 있으므로, 항상 여러 방법을 실험해봐야 합니다.

4 실습/코드: 텍스트 분류 모델 구현

이 섹션에서는 *20 Newsgroups* 데이터셋을 사용하여 텍스트 분류 모델을 구현하는 과정을 단계별로 보여줍니다. 토큰화, 어간 추출, 표제어 추출을 적용한 세 가지 시나리오를 비교합니다.

과제 2번 및 3번 관련 힌트

과제 2번은 'Bag of Words' 방식을 사용하여 텍스트 분류를 수행합니다. 이는 시간 정보를 잃지만, 특정 분류 문제에서 효율적일 수 있습니다. 과제 3번은 순환 신경망을 사용하여 텍스트 분류를 수행하며, 본 실습과 유사한 접근 방식을 사용합니다.

4.1 환경 설정 및 데이터 로딩

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import fetch_20newsgroups
4 from tensorflow.keras.preprocessing.text import Tokenizer
5 from tensorflow.keras.preprocessing.sequence import pad_sequences
6 from tensorflow.keras.models import Sequential
7 from tensorflow.keras.layers import Embedding, LSTM, Dropout, Dense
8 from nltk.tokenize import word_tokenize
9 from nltk.stem import PorterStemmer, SnowballStemmer, WordNetLemmatizer
10 # from nltk.corpus import wordnet # WordNetLemmatizer 사용시필요
11 # import spacy # SpaCy 사용시필요
12
13 # NLTK 데이터다운로드최초 ( 회1 실행)
14 # nltk.download('punkt')
15 # nltk.download('wordnet')
16 # nltk.download('omw-1.4')
17 # nltk.download('averaged_perceptron_tagger')
18
19 # 20 Newsgroups 데이터셋로드
20 categories = ['rec.sport.hockey', 'misc.forsale']
21 train_data = fetch_20newsgroups(subset='train', categories=categories, shuffle=True, random_state=42)
22 test_data = fetch_20newsgroups(subset='test', categories=categories, shuffle=True, random_state=42)
23
24 train_texts = train_data.data
25 train_labels = train_data.target
26 test_texts = test_data.data
27 test_labels = test_data.target
28
29 print(f"훈련 데이터개수 : {len(train_texts)}")
30 print(f"테스트 데이터개수 : {len(test_texts)}")
31 print(f"카테고리: {train_data.target_names}")
32
33 # 예시데이터확인
34 print("\n--- 훈련데이터예시 ---")

```

```

35 print(f"텍스트 번째 (53): {train_texts[52][:200]}...")
36 print(f"레이블 번째 (53): {train_labels[52]} ({train_data.target_names[
    train_labels[52]])} ") # 0: for sale
37 print(f"텍스트 번째 (65): {train_texts[64][:200]}...")
38 print(f"레이블 번째 (65): {train_labels[64]} ({train_data.target_names[
    train_labels[64]])} ") # 1: hockey

```

Listing 5: 환경 설정 및 데이터 로딩

4.2 모델 정의 함수

앞서 설명한 *LSTM* 기반 모델 아키텍처를 함수로 정의합니다.

```

1 def build_lstm_model(max_features, embedding_dim, max_len):
2     model = Sequential()
3     model.add(Embedding(input_dim=max_features + 1, output_dim=embedding_dim,
4         input_length=max_len))
5     model.add(Dropout(0.2))
6     model.add(LSTM(128))
7     model.add(Dropout(0.2))
8     model.add(Dense(1, activation='sigmoid'))
9     model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['
    accuracy'])
10    return model

```

Listing 6: 텍스트 분류 모델 정의 함수

4.3 시나리오 1: 토큰화만 적용

텍스트를 단어 토큰으로만 분할하고, 어간 추출이나 표제어 추출은 적용하지 않습니다.

```

1 # 전처리파라미터
2 max_features = 10000 # 가장빈번한개 10000 단어사용
3 max_len = 500 # 시퀀스최대길이
4 embedding_dim = 128 # 임베딩벡터차원
5
6 # 1. 토큰화및단어인덱스생성
7 # Keras 는Tokenizer 자동으로단어빈도를계산하고인덱스를부여합니다
8 # 는num_words max_features + 1 (OOV 포함)
9 tokenizer = Tokenizer(num_words=max_features, oov_token="<OOV>")
10 tokenizer.fit_on_texts(train_texts)
11
12 # 텍스트를인덱스시퀀스로변환
13 train_sequences = tokenizer.texts_to_sequences(train_texts)
14 test_sequences = tokenizer.texts_to_sequences(test_texts)
15
16 # 시퀀스패딩최대 ( 길이맞추기 )
17 train_padded = pad_sequences(train_sequences, maxlen=max_len, padding='post',
    truncating='post')

```

```

18 test_padded = pad_sequences(test_sequences, maxlen=max_len, padding='post',
    truncating='post')
19
20 print("\n--- 시나리오 1: 토큰화만적용 ---")
21 print(f"훈련 데이터패딩후형태 : {train_padded.shape}")
22 print(f"테스트 데이터패딩후형태 : {test_padded.shape}")
23 print(f"단어 사전크기 : {len(tokenizer.word_index)}") # 실제단어인덱스수
24
25 # OOV 예시강의에서 ( 언급된 philip_ws@spk.hp.com)
26 # 이메일주소는빈번하지않아로 OOV 처리될가능성이높음
27 example_text = "From: philip_ws@spk.hp.com (Philip W. S)"
28 example_sequence = tokenizer.texts_to_sequences([example_text])
29 print(f"예시 텍스트: '{example_text}'")
30 print(f"변환된 시퀀스 (OOV 포함): {example_sequence}")
31 # 출력에서는 0 OOV 토큰을의미합니다 .
32
33 # 2. 모델구축및훈련
34 model_tokenize = build_lstm_model(max_features, embedding_dim, max_len)
35 print("\n--- 모델요약토큰화만 () ---")
36 model_tokenize.summary()
37
38 history_tokenize = model_tokenize.fit(
39     train_padded, train_labels,
40     epochs=10,
41     batch_size=32,
42     validation_split=0.2 # 훈련데이터의를 20% 검증데이터로사용
43 )
44
45 # 3. 모델평가
46 loss_tokenize, accuracy_tokenize = model_tokenize.evaluate(test_padded,
    test_labels)
47 print(f"\테스트n 정확도토큰화만 (): {accuracy_tokenize:.4f}")
48
49 # 결과시각화
50 plt.figure(figsize=(12, 4))
51 plt.subplot(1, 2, 1)
52 plt.plot(history_tokenize.history['accuracy'], label='Train Accuracy')
53 plt.plot(history_tokenize.history['val_accuracy'], label='Validation Accuracy'
    )
54 plt.title('Accuracy (Tokenization Only)')
55 \xlabel('Epoch')
56 \ylabel('Accuracy')
57 \legend()
58
59 \subplot(1, 2, 2)
60 \plot(history_tokenize.history['loss'], label='Train Loss')
61 \plot(history_tokenize.history['val_loss'], label='Validation Loss')
62 \title('Loss (Tokenization Only)')
63 \xlabel('Epoch')
64 \ylabel('Loss')

```

```

65 \legend()
66 \tight_layout()
67 \show()

```

Listing 7: 시나리오 1: 토큰화만 적용

4.4 시나리오 2: 토큰화 + 어간 추출 적용

텍스트를 토큰화한 후, *Porter Stemmer*를 사용하여 각 단어의 어간을 추출합니다.

```

1  porter_stemmer = PorterStemmer()
2
3  def stem_text(text):
4      tokens = word_tokenize(text)
5      stemmed_tokens = [porter_stemmer.stem(token) for token in tokens]
6      return " ".join(stemmed_tokens)
7
8  # 훈련및테스트텍스트에어간추출적용
9  train_texts_stemmed = [stem_text(text) for text in train_texts]
10 test_texts_stemmed = [stem_text(text) for text in test_texts]
11
12 print("\n--- 시나리오 2: 토큰화 + 어간추출적용 ---")
13 # 예시텍스트확인강의에서 ( 언급된 octopus -> octop)
14 example_original = train_texts[64][:200]
15 example_stemmed = stem_text(example_original)
16 print(f"원본 텍스트번째 (65): {example_original}...")
17 print(f"어간 추출후텍스트 : {example_stemmed}...")
18 # 'octopus가' 'octop으로', 'years가' 'year로' 변환되는것을볼수있습니다
19
20 # 1. 토큰화및단어인덱스생성어간 ( 추출된텍스트사용 )
21 tokenizer_stem = Tokenizer(num_words=max_features, oov_token="<OOV>")
22 tokenizer_stem.fit_on_texts(train_texts_stemmed)
23
24 train_sequences_stem = tokenizer_stem.texts_to_sequences(train_texts_stemmed)
25 test_sequences_stem = tokenizer_stem.texts_to_sequences(test_texts_stemmed)
26
27 train_padded_stem = pad_sequences(train_sequences_stem, maxlen=max_len,
28                                   padding='post', truncating='post')
29
30 test_padded_stem = pad_sequences(test_sequences_stem, maxlen=max_len, padding=
31                                  'post', truncating='post')
32
33 print(f"훈련 데이터패딩후형태어간 ( 추출): {train_padded_stem.shape}")
34 print(f"단어 사전크기어간 ( 추출): {len(tokenizer_stem.word_index)}")
35
36 # 2. 모델구축및훈련
37 model_stem = build_lstm_model(max_features, embedding_dim, max_len)
38 print("\n--- 모델요약어간 ( 추출) ---")
39 model_stem.summary()
40
41 history_stem = model_stem.fit(

```

```

39     train_padded_stem, train_labels,
40     epochs=10,
41     batch_size=32,
42     validation_split=0.2
43 )
44
45 # 3. 모델평가
46 loss_stem, accuracy_stem = model_stem.evaluate(test_padded_stem, test_labels)
47 print(f"\테스트n 정확도어간 ( 추출): {accuracy_stem:.4f}")
48
49 # 결과시각화
50 plt.figure(figsize=(12, 4))
51 plt.subplot(1, 2, 1)
52 plt.plot(history_stem.history['accuracy'], label='Train Accuracy')
53 plt.plot(history_stem.history['val_accuracy'], label='Validation Accuracy')
54 \title('Accuracy (Stemming)')
55 \xlabel('Epoch')
56 \ylabel('Accuracy')
57 \legend()
58
59 \subplot(1, 2, 2)
60 \plot(history_stem.history['loss'], label='Train Loss')
61 \plot(history_stem.history['val_loss'], label='Validation Loss')
62 \title('Loss (Stemming)')
63 \xlabel('Epoch')
64 \ylabel('Loss')
65 \legend()
66 \tight_layout()
67 \show()

```

Listing 8: 시나리오 2: 토큰화 + 어간 추출 적용

4.5 시나리오 3: 토큰화 + 표제어 추출 적용

텍스트를 토큰화한 후, *NLTK*의 *WordNetLemmatizer*를 사용하여 각 단어의 표제어를 추출합니다. (품사 태깅이 필요하지만, 여기서는 단순화를 위해 동사로 가정합니다.)

```

1 lemmatizer = WordNetLemmatizer()
2
3 def lemmatize_text(text):
4     tokens = word_tokenize(text)
5     # 실제사용시에는 nltk.를 pos_tag 사용하여 각 단어의 정확한 품사를 파악해야합니
      다
6     # 여기서는 단순화를 위해 모든 단어를 동사 (v)로 가정합니다.
7     lemmatized_tokens = [lemmatizer.lemmatize(token, pos='v') for token in
      tokens]
8     return " ".join(lemmatized_tokens)
9
10 # 훈련 및 테스트 텍스트에 표제어 추출 적용
11 train_texts_lemmatized = [lemmatize_text(text) for text in train_texts]

```

```

12 test_texts_lemmatized = [lemmatize_text(text) for text in test_texts]
13
14 print("\n--- 시나리오 3: 토큰화 + 표제어추출적용 ---")
15 # 예시텍스트확인강의에서 ( 언급된 driving -> drive)
16 example_text_drive = "I was driving my car."
17 example_lemmatized_drive = lemmatize_text(example_text_drive)
18 print(f"원본 텍스트: '{example_text_drive}'")
19 print(f"표제어 추출후텍스트 : '{example_lemmatized_drive}'")
20 # 'driving'이 'drive'로 변환되는것을볼수있습니다 .
21
22 # 1. 토큰화및단어인덱스생성표제어 ( 추출된텍스트사용 )
23 tokenizer_lem = Tokenizer(num_words=max_features, oov_token="<OOV>")
24 tokenizer_lem.fit_on_texts(train_texts_lemmatized)
25
26 train_sequences_lem = tokenizer_lem.texts_to_sequences(train_texts_lemmatized)
27 test_sequences_lem = tokenizer_lem.texts_to_sequences(test_texts_lemmatized)
28
29 train_padded_lem = pad_sequences(train_sequences_lem, maxlen=max_len, padding=
    'post', truncating='post')
30 test_padded_lem = pad_sequences(test_sequences_lem, maxlen=max_len, padding='
    post', truncating='post')
31
32 print(f"훈련 데이터패딩후형태표제어 ( 추출): {train_padded_lem.shape}")
33 print(f"단어 사전크기표제어 ( 추출): {len(tokenizer_lem.word_index)}")
34
35 # 2. 모델구축및훈련
36 model_lem = build_lstm_model(max_features, embedding_dim, max_len)
37 print("\n--- 모델요약표제어 ( 추출) ---")
38 model_lem.summary()
39
40 history_lem = model_lem.fit(
41     train_padded_lem, train_labels,
42     epochs=10,
43     batch_size=32,
44     validation_split=0.2
45 )
46
47 # 3. 모델평가
48 loss_lem, accuracy_lem = model_lem.evaluate(test_padded_lem, test_labels)
49 print(f"\테스트n 정확도표제어 ( 추출): {accuracy_lem:.4f}")
50
51 # 결과시각화
52 plt.figure(figsize=(12, 4))
53 plt.subplot(1, 2, 1)
54 plt.plot(history_lem.history['accuracy'], label='Train Accuracy')
55 plt.plot(history_lem.history['val_accuracy'], label='Validation Accuracy')
56 \title('Accuracy (Lemmatization)')
57 \xlabel('Epoch')
58 \ylabel('Accuracy')
59 \legend()

```



```
60
61 \subplot(1, 2, 2)
62 \plot(history_lem.history['loss'], label='Train Loss')
63 \plot(history_lem.history['val_loss'], label='Validation Loss')
64 \title('Loss (Lemmatization)')
65 \xlabel('Epoch')
66 \ylabel('Loss')
67 \legend()
68 \tight_layout()
69 \show()
```

Listing 9: 시나리오 3: 토큰화 + 표제어 추출 적용

5 체크리스트

본 문서를 학습하고 과제를 수행하며 점검할 수 있는 체크리스트입니다.

학습 점검 체크리스트

- *RNN* 기본 이해: 순환 신경망이 시퀀스 데이터를 어떻게 처리하는지 이해했는가?
- *RNN* 파라미터 계산: *SimpleRNN* 레이어의 파라미터 수를 직접 계산할 수 있는가?
- *LSTM* 작동 원리: *LSTM*의 셀 상태(*C*)와 은닉 상태(*H*), 그리고 게이트 메커니즘을 설명할 수 있는가? *C*와 *H*의 차원이 왜 같은지 이해했는가?
- 양방향 *RNN*: 양방향 *RNN*이 필요한 이유와 파라미터/출력 차원 변화를 이해했는가?
- *NLP* 개요: *NLP*의 주요 응용 분야와 도전 과제를 설명할 수 있는가?
- 텍스트 전처리: 토큰화, 어간 추출, 표제어 추출의 정의, 목적, 차이점을 명확히 구분할 수 있는가? 각 기법의 장단점은 무엇인가?
- 임베딩 레이어: 임베딩 레이어가 왜 필요하며, 원-핫 인코딩의 한계를 어떻게 극복하는지 설명할 수 있는가?
- 드롭아웃: 드롭아웃이 과적합을 어떻게 방지하는지 이해했는가?
- 손실/활성화 함수: 이진 분류 문제에서 시그모이드와 이진 교차 엔트로피를 사용하는 이유를 설명할 수 있는가?

과제 수행 체크리스트

- 데이터 로딩: *20 Newsgroups* 데이터셋을 올바르게 로드하고 훈련/테스트 데이터로 분리했는가?
- 텍스트 전처리 구현: 토큰화, 어간 추출, 표제어 추출 함수를 정확히 구현했는가?
- *OOV* 처리: *OOV* 토큰을 포함하여 단어 인덱싱을 올바르게 처리했는가?
- 시퀀스 패딩: $\text{pad}_{sequences}$?
- 모델 구축: *Embedding*, *LSTM*, *Dropout*, *Dense* 레이어를 사용하여 모델을 올바르게 정의했는가?
- 모델 훈련 및 평가: 모델을 훈련하고 테스트 데이터셋에서 정확도를 평가했는가?
- 결과 분석: 각 전처리 기법(토큰화만, 어간 추출, 표제어 추출)에 따른 모델 성능 변화를 분석하고 설명할 수 있는가?

6 FAQ (자주 묻는 질문)

Q1: RNN 파라미터 계산은 어떻게 하나요?

A1: SimpleRNN(은닉 뉴런 수, $input_shape = (None, \quad)$) , (입력 특성 수 * 은닉 뉴런 수)

Q2: LSTM의 셀 상태(C)와 은닉 상태(H)는 왜 같은 차원인가요?

A2: LSTM 내부에서 C 와 H 는 요소별 덧셈(*summation*)과 요소별 곱셈(*element-wise multiplication*) 연산을 통해 상호작용합니다. 이러한 연산들은 벡터의 차원을 변경하지 않으므로, C 와 H 는 항상 동일한 차원을 유지해야 합니다.

Q3: 양방향 RNN은 왜 필요한가요?

A3: 일반 RNN은 과거 정보만으로 현재를 예측하지만, 텍스트와 같은 시퀀스 데이터에서는 미래 정보(문장의 뒷부분)가 현재의 의미를 이해하는 데 중요할 수 있습니다. 양방향 RNN은 순방향과 역방향으로 모두 학습하여 과거와 미래의 모든 문맥 정보를 활용하여 더 정확한 예측을 가능하게 합니다.

Q4: 어간 추출(*Stemming*)과 표제어 추출(*Lemmatization*)의 차이는 무엇인가요?

A4: 어간 추출은 단어의 접미사를 단순히 잘라내어 '어간'을 찾습니다. 빠르지만 결과가 실제 사전에 없는 단어일 수 있습니다 (예: 'driving' -> 'driv'). 표제어 추출은 단어의 품사 정보를 활용하여 사전적 의미를 가지는 '표제어'를 찾습니다. 느리지만 결과가 항상 실제 단어입니다 (예: 'driving' -> 'drive').

Q5: 임베딩 레이어는 왜 사용하나요?

A5: 임베딩 레이어는 고차원의 희소(*sparse*)한 원-핫 인코딩의 비효율성을 해결하고, 단어 간의 의미적 유사성을 저차원의 밀집(*dense*) 벡터 공간에 반영하기 위해 사용됩니다. 이는 모델의 학습 효율과 성능을 높이는 데 기여합니다.

Q6: OOV(*Out-Of-Vocabulary*) 단어는 어떻게 처리하나요?

A6: OOV 단어는 모델 학습 시 사용된 어휘 사전에 없는 단어를 의미합니다. 일반적으로 OOV 토큰을 위한 특별한 인덱스(예: 0)를 사전에 할당하고, 사전에 없는 단어가 나타나면 이 OOV 인덱스로 대체합니다. 이는 모델이 새로운 단어를 만나도 처리할 수 있도록 합니다.

Q7: 드롭아웃(*Dropout*)은 왜 사용하나요?

A7: 드롭아웃은 신경망의 과적합을 방지하기 위한 정규화 기법입니다. 훈련 과정에서 무작위로 일부 뉴런의 연결을 끊어 모델이 특정 특징에 과도하게 의존하는 것을 막고, 다양한 하위 네트워크를 학습하는 효과를 통해 모델의 일반화 성능을 향상시킵니다.

7 빠르게 훑어보기 (1페이지 요약)

CSCI89 Lecture 3 핵심 요약

1. 순환 신경망 (*RNN*)의 이해
 - 시퀀스 데이터 처리 특화 (텍스트, 음성, 시계열).
 - 이전 정보 기억 (은닉 상태).
 - 파라미터 계산: (입력 특성 수 + 은닉 뉴런 수 + 1) * 은닉 뉴런 수.
 - 입출력 유형: 다대다, 다대일, 일대다 (*Keras return_sequences*).
2. 장단기 기억망 (*LSTM*)
 - *RNN*의 장기 의존성 문제 해결.
 - 두 가지 기억: 셀 상태 (*C*, 장기 기억)와 은닉 상태 (*H*, 단기 기억).
 - 게이트 메커니즘: 망각, 입력, 출력 게이트로 정보 흐름 제어.
 - *C*와 *H* 차원: 항상 동일. *LSTM(N)*은 내부적으로 $4 \times N$ 뉴런.
3. 양방향 *RNN/LSTM*
 - 순방향/역방향 시퀀스 동시 처리로 전체 문맥 활용.
 - 파라미터 수: 단방향의 2배.
 - 출력 차원: 단방향의 2배 (연결 (*concatenate*) 때문).
4. 자연어 처리 (*NLP*) 개요
 - 컴퓨터가 인간 언어를 이해/생성하는 *AI* 분야.
 - 도전 과제: 언어의 모호성, 문맥, 배경 지식 필요.
 - 주요 응용: 텍스트 분류, *NER*, 감성 분석, 기계 번역, 챗봇 등.
5. 텍스트 전처리 기법
 - 토큰화: 텍스트를 의미 단위(단어, 문장 등)로 분할. 불용어 처리.
 - 어간 추출 (*Stemming*): 단어의 접미사 제거 (예: '*running*' → '*runn*'). 빠르지만 실제 단어가 아닐 수 있음. 차원 축소.
 - 표제어 추출 (*Lemmatization*): 단어를 사전형(표제어)으로 변환 (예: '*driving*' → '*drive*'). 정확하지만 품사 정보 필요, 비용 높음.
6. 텍스트를 숫자로: 임베딩 (*Embedding*)
 - 고차원 희소(*Sparse*) 원-핫 인코딩의 한계 극복.
 - 단어 인덱스를 저차원 밀집(*Dense*) 벡터로 변환.
 - 단어 간 의미적 유사성 학습.
 - *OOV* (*Out-Of-Vocabulary*): 사전에 없는 단어 처리 (특별 인덱스 할당).
7. 모델 정규화: 드롭아웃 (*Dropout*)
 - 훈련 중 무작위로 뉴런 연결 끊어 과적합 방지.
 - 모델의 일반화 성능 향상.
8. 텍스트 분류 모델 (*LSTM* 기반)